

Void Sower: Bao Orbital Batteries

Technical Design and Architecture Specification

Ludic Framework and Thematic Game Design

The mobile shoot 'em up and arcade shooter genres have long relied on reactive twitch reflexes, high-density projectile dodging, and incremental numerical gear treadmills¹. These conventions frequently experience rapid player retention decay because core encounters lack combinatorial depth and structural variation¹. *Void Sower: Bao Orbital Batteries* resolves this design stagnation by fusing the mathematical rigor, count-and-capture logic, and cascading momentum of the traditional East African board game *Bao la Kiswahili* with a vertical orbital defense shooter¹.

Thematic worldbuilding roots this synthesis within an Afrofuturist space opera setting across the Kilwa Nebula Basin, an interstellar maritime expanse directly inspired by historical Swahili Coast trading bastions¹. Planetary systems are linked by high-velocity tachyon streams designated as the Monsoon Lanes¹. The player commands a stationary orbital dreadnought defending an atmospheric siphon against the Silent Maw, an automated harvesting armada descending through structured geometric attack corridors¹. In contrast to standard shooters that feature continuous blaster fire, *Void Sower* removes traditional fire buttons entirely¹. Offensive output is driven by discrete, deterministic resource distributions mapped directly from the *namua* (placement) and *mtaji* (capture and relay) mechanics of *Bao*¹.

```
C++
// Architectural definition of the 16-bay dual-tier dreadnought ring buffer
enum class BatteryTier : uint8_t
{
    InnerReservoir = 0 // Bays 0 - 7: High-capacity storage and internal relay routing
    OuterFrontline = 1 // Bays 8 - 15: Direct corridor alignment and particle lance emission
}
```

The defensive superstructure of the dreadnought features sixteen plasma capacitor bays arranged in two concentric tiers: an inner reservoir ring consisting of bays 0 through 7, and an outer frontline battery ring consisting of bays 8 through 15¹. Assault vessels advance downward through eight vertical attack corridors that align with the eight outer frontline bays¹.

The player loads discrete plasma cores from a central reactor into the battery ring¹. Depositing a core into an occupied chamber scoops all contained plasma units and distributes them one by

one into consecutive chambers along a chosen clockwise or counter-clockwise trajectory¹. Combat interactions resolve at the termination of the distribution: landing in a frontline battery aligned with an active hostile corridor triggers an axial cross-discharge particle lance¹. Landing in a bay that already contains charge initiates an automated relay sow, scooping the combined mass and continuing traversal while discharging secondary radial flak detonations¹. This transforms real-time space defense into a forward-calculating spatial puzzle where a single core placement can trigger multi-lap cascades that sweep entire battle fleets¹.

Traditional Bao la Kiswahili Mechanic	Void Sower: Bao Orbital Batteries System Translation	Tactical Combat Behavior and Mechanical Function
Namua Phase (Reserve piece placement into active pits) ¹ .	Plasma Core Injection: Upward flick gesture loads a discrete core from the central reactor into an active bay ¹ .	Initiates the combat turn. Increments target bay mass by $+1$, then empties the chamber to begin circular ring traversal ¹ .
Mtaji Capture (Final seed lands in an occupied frontline pit facing opponent pieces) ¹ .	Frontline Cross-Discharge: Sowing termination in an occupied outer frontline bay aligned with a hostile corridor ¹ .	Fires an axial particle lance that pierces heavy armor and barriers, dealing quadratic damage based on accumulated mass ¹ .
Relay Sowing (Landing in an occupied pit scoops contents to sustain distribution) ¹ .	Capacitor Overload Relay: Final seed lands in any bay possessing an existing charge of $M \geq 1$.	Automatically scoops combined energy payload ($M +$) and continues ring traversal while emitting radial flak bursts ¹ .
Takata Move (Non-capturing sow executed when captures are unavailable) ³ .	Tactical Charge Redistribution: Sowing termination in an unaligned or non-frontline chamber ¹ .	No primary lance discharges; accumulated mass remains banked across internal capacitors to set up subsequent high-mass strikes ¹ .
Kichwa (Terminal "head" pits that determine relay entry direction) ³ .	Vector Conduits (Bays 8 & 15): Lateral edge batteries flanking the outer hull ¹ .	Allows the player to reverse or re-route angular momentum across tiers

		without kinetic energy degradation ¹ .
Kimbi (Flank pits adjacent to heads enforcing directional shifts) ³ .	Deflection Chambers (Bays 9 & 14): Angular structural capacitor nodes ¹ .	Triggering a cross-discharge from deflection bays forces subsequent secondary flak arcs into opposite directional vectors ¹ .
Nyumba ("House" pit; specialized reservoir with strategic retention rules) ³ .	Central Super-Capacitor (Bays 3 & 4): Protected core reservoir bays ¹ .	Banks large quantities of plasma mass without auto-relaying, acting as an emergency reserve until manually tapped ¹ .

The primary damage inflicted by the particle lance scales quadratically with respect to the total plasma mass concentrated in the terminal frontline bay¹. Letting M denote the accumulated plasma units within the discharging chamber, and α represent the baseline beam focal coefficient, total lance damage $D(M)$ is defined as:

$$D(M) = \alpha \cdot M^2$$

This quadratic relationship establishes an explicit tactical incentive: firing a single isolated core yields an output of α , whereas coordinating a multi-lap relay cascade that concentrates eight units into a terminal frontline strike yields 64α ¹. Hostile dreadnoughts and armored carriers project hardened deflector barriers that completely absorb low-energy fire, requiring the player to engineer high-mass concentrations through calculated relay paths¹. When a sowing sequence deposits its final unit into an occupied bay, the capacitor enters an overload state, scooping the combined mass $M' = M_{existing} + 1$ to continue distribution¹. As this relay cycles through intermediate chambers, the platform vents excess thermal energy into the atmosphere, triggering radial flak bursts:

$$D_{flak} = \beta \cdot \sqrt{M'}$$

where β is the sub-munition dispersion coefficient¹. These rolling flak detonations destroy swarms of light fighters and drone screens in adjacent corridors¹.

The system relies on mathematical mass conservation: because cross-discharges completely empty the firing chamber, and non-capturing distributions dilute plasma into empty chambers, energy states remain finite and strictly bounded⁸. Cascades cannot trigger infinite loops; traversal terminates deterministically the moment the final seed drops into a depleted bay (

$$M_{\text{destination}} == 0)^2.$$

Tactical failure occurs under two conditions: if descending enemy craft cross the planetary orbital boundary ($y \leq Y_{\text{boundary}}$), or if the dreadnought exhausts its finite store of unplaced plasma cores prior to neutralizing the assault wave¹. Sector victory requires eliminating all hostile combatants, culminating in the destruction of the sector flagship whose multi-corridor kinetic barriers require simultaneous, multi-battery cross-discharges to shatter¹.

Technical Architecture and Engine Integration

To achieve optimal performance on mobile hardware while retaining user interface flexibility, *Void Sower* uses the technical stack established during the development of *Oware 2048*: a C++ engine backend integrated with Flutter and the Fluorite engine framework². The architecture establishes strict separation between state evaluation and hardware-accelerated rendering². The Flutter framework functions as the presentation and orchestration layer, managing metagame navigation, star maps, dialog sequences, and monetization SDKs². The 3D space combat arena is rendered via the FluoriteView widget, which embeds a native C++ rendering pipeline directly into the Flutter layout hierarchy².

Graphics processing is executed by Google's Filament renderer, targeting Vulkan 1.1+ on modern Android devices and falling back to OpenGL ES 3.1 on legacy hardware². Filament delivers console-grade physically based rendering (PBR) and post-processing with low thermal overhead and minimal draw-call submission latency².

Communication between Flutter and the C++ engine core completely bypasses Flutter's standard MethodChannel⁸. Traditional platform channels serialize structured data into JSON or binary buffers and traverse multiple thread contexts, generating latency ($10\text{--}30\text{ ms}$) and triggering garbage collection spikes that disrupt high-speed cascades⁸.

Void Sower utilizes Dart Foreign Function Interface (FFI) to establish direct, zero-copy shared-memory pointers between the Dart runtime and the compiled C++ engine core (libvoid_sower_core.so / libvoid_sower_core.dylib)⁸. Both runtimes access identical memory addresses in RAM⁸. The C++ backend exposes a pure C Application Binary Interface (ABI) via extern "C", with typed Dart bindings automatically synthesized using Flutter's ffigen tool⁸.

Architectural Domain	Technical Specification	Operational Implementation and Performance Role
----------------------	-------------------------	---

Presentation & UI	Flutter 3.x Framework	Manages touch input capture, accessibility, HUD layering, and campaign state ² .
3D Rendering Subsystem	Google Filament & FluoriteView	Embedded C++ PBR graphics pipeline running on Vulkan 1.1+ and OpenGL ES 3.1 ² .
Runtime Data Bridge	Dart Foreign Function Interface (FFI)	Zero-copy shared-memory pointers; sub-millisecond ABI invocation without IPC serialization ⁸ .
ECS Simulation Engine	EnTT C++ Framework	Cache-aligned Plain Old Data components stored contiguously in sparse set arrays ² .
Memory Management	Pre-allocated Object Pools	Fixed contiguous buffers for beams, flak bursts, and projectiles, eliminating runtime heap allocations ² .
Collision Subsystem	2D Uniform Spatial Grid	Bucketed spatial partitioning reducing beam-to-ship intersection tests from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ ² .
Monetization & Analytics	UMP SDK, Firebase, GameAnalytics	Dual telemetry tracking ARPDAU and difficulty drop-offs; privacy-compliant rewarded interstitials ² .

Runtime memory management operates under a zero-allocation constraint during active combat loops². The dynamic instantiation of projectile entities, particle systems, and raycast colliders through standard heap allocators (malloc, new) causes heap fragmentation and triggers operating system garbage collection pauses².

Void Sower pre-allocates hundreds of entity records within contiguous memory pools during initial sector load times². Reusable entities such as particle lance segments, flak bursts, and impact flashes are recycled using EnTT component tagging (Tag_Active, Tag_Available),

maintaining zero dynamic allocation throughout the frame lifecycle².

Particle lances and atmospheric energy discharges are rendered via custom shaders authored in Filament's shading language and pre-compiled offline into binary packages using the matc compiler tool¹³. Dynamic beam pulsation, heat bloom, and electromagnetic noise are parameterized via GPU uniform buffers updated synchronously from ECS components, enabling visual feedback without requiring dynamic vertex recalculations on the CPU¹².

Data-Oriented ECS and Deterministic Simulation Core

The computational backbone of *Void Sower* is implemented in C++ using the EnTT Entity-Component-System framework². Rather than organizing logic around traditional object-oriented ship or bullet hierarchies, the architecture follows Data-Oriented Design principles⁸. Entities are lightweight numeric identifiers that index into contiguous arrays of Plain Old Data (POD) components packed inside sparse sets⁸.

This ensures that when simulation systems iterate across active batteries or traversing sows, sequential memory addresses are pre-fetched directly into the CPU's L1 cache line, eliminating cache misses and maximizing instruction throughput on mobile ARM architectures⁸.

```
C++
```

```
#pragma pack(push, 1)
```

```
// Identifies and configures individual capacitor bays on the hull
```

```
struct BatteryComponent {
```

```
    uint8_t bay_index;        // 0 to 15
```

```
    uint8_t tier;              // 0: Inner Reservoir, 1: Outer Frontline
```

```
    uint16_t end_column;       // Assigned geometric screen corridor
```

```
    uint32_t charge_units;     // Accumulated plasma mass (M)
```

```
    float radial_position_rad;  // Geometric angle on dreadnought chassis
```

```
    bool is_frontline;         // Capable of direct cross-discharge
```

```
}
```

```
// Tracks active sowing traversals and multi-lap cascade state
```

```
struct SowingStateComponent {
```

```
    uint8_t origin_bay;        // Sowing commencement bay
```

```
    uint8_t current_bay;       // Dynamic position of distribution head
```

```
    uint32_t remaining_units;   // Plasma units remaining to sow
```

```
    int8_t step_direction;      // +1 for Clockwise, -1 for Counter-Clockwise
```

```
    float step_accumulator;     // Interpolation accumulator for frame pacing
```

```
    uint16_t cascade_depth;     // Relay lap tracking counter
```

```
}
```

```

// Encapsulates descending enemy assault craft
struct EnemyVesselComponent {
    uint32_t entity_id; // Unique combatant handle
    uint16_t assigned_corridor; // Attack corridor matching frontline bay
    float world_pos_x; // Spatial coordinate X
    float world_pos_y; // Spatial coordinate Y
    float velocity_y; // Advance speed toward atmospheric boundary
    float current_shields; // Quadratic damage mitigation buffer
    float max_shields; // Base barrier capacity
    float current_hull; // Structural integrity points
    float max_hull; // Base hull capacity
    uint8_t vessel_type; // 0: Escort, 1: Cruiser, 2: Flagship
}

```

```

// Configures active high-energy particle lance discharges
struct ParticleLanceComponent {
    uint8_t firing_bay_index; // Originating frontline battery
    float origin_x; // Beam origin X
    float origin_y; // Beam origin Y
    float beam_width; // Dynamic visual width based on mass
    float sustained_duration; // Firing duration in seconds
    float remaining_duration; // Life timer countdown
    float total_damage; // Quadratic damage value ( $\alpha * M^2$ )
    bool active; // Simulation status flag
}

```

```

// Represents secondary radial flak detonations
struct FlakBurstComponent {
    float world_pos_x; // Detonation coordinate X
    float world_pos_y; // Detonation coordinate Y
    float blast_radius; // Effective sub-munition dispersion radius
    float area_damage; // Secondary damage value ( $\beta * \sqrt{M'}$ )
    float lifetime; // Fadeout duration in seconds
}

```

```

// Global single-instance state tracking dreadnought systems
struct DreadnoughtStateComponent {
    float orbital_position_x; // Platform position along defense shelf
    float target_position_x; // Smoothed interpolation target
    uint32_t reserve_cores; // Unplaced core inventory
    float boundary_line_y; // Critical planetary atmospheric threshold
    bool s_cascading; // Input lock flag during active relays
}

```

```
#pragma pack(pop)
```

The sixteen battery chambers map to an array index range $i \in [0, 15]$ ². Clockwise and counter-clockwise traversals are governed by circular ring buffer arithmetic utilizing modulo 16 operations². Because standard C++ truncated division does not evaluate negative dividends to positive mathematical modulo residues, directional transitions enforce modulo arithmetic via:

```
C++
inline uint8_t StepBayIndex(uint8_t current_index, int8_t direction)
{
    return static_cast<uint8_t>((current_index + direction * 16) % 16);
}
```

Evaluating multi-step trajectories during predictive targeting calculations occurs in constant time $\mathcal{O}(1)$, eliminating iteration loops and conditional boundary checking during cascade evaluation²:

$$i' = (i + \text{direction} \cdot k) \pmod{16}$$

Runtime execution is governed by a deterministic Finite State Machine (FSM) running at a fixed 60 Hz simulation rate, ensuring repeatable state calculations⁸.

State Identifier	Entry Condition	Internal System Logic Execution	Exit Transition Logic
STATE_ORBITAL_IDLE	Level load complete or previous cascade resolved ⁸ .	Physics systems advance hostile flight vectors; HUD renders prospective targeting reticles; engine listens for FFI touch callbacks ⁸ .	Transitions to STATE_CORE_INJECTION upon registration of an upward flick gesture ¹ .
STATE_CORE_INJECTION	Player flick gesture validated ¹ .	Target bay decrements reserve inventory by 1; creates	Automatically transitions to STATE_SOWING_TRAVERSAL on next

		SowingStateComponent with remaining_units = charge_units + 1; zeroes origin bay charge ¹ .	tick ⁸ .
STATE_SOWING_T RAVERSAL	SowingStateComponent is attached and active ⁸ .	Steps current_bay modulo 16; deposits $+1$ mass; decrements remaining_units; instantiates flak trail entities along the hull ¹ .	Transitions to STATE_EVALUATE _DESTINATION when remaining_units == 0 ⁸ .
STATE_EVALUATE _DESTINATION	Distribution phase concludes ⁸ .	Inspects terminal bay; if frontline and aligned with hostile corridor, flags cross-discharge; if destination already held charge, flags auto-relay ¹ .	Transitions to STATE_CROSS_DI SCHARGE if aligned; else to STATE_RELAY_OV ERLOAD if $M \geq$; else transitions to STATE_CLEANUP_ CHECK ⁸ .
STATE_CROSS_DI SCHARGE	Terminal frontline alignment verified ¹ .	Spawns ParticleLanceComponent; computes damage $D = \alpha \cdot M^2$; executes corridor raycast; clears bay charge ¹ .	Transitions to STATE_RELAY_OV ERLOAD if remaining cascade exists, or STATE_CLEANUP_ CHECK when beam expires ⁸ .
STATE_RELAY_OV ERLOAD	Destination bay held prior charge ¹ .	Scoops total terminal mass; updates SowingStateComponent	Recursively transitions back to STATE_SOWING_T RAVERSAL ⁸ .

		nent; increments cascade_depth; fires flak detonations into adjacent corridors ¹ .	
STATE_CLEANUP_CHECK	Traversal terminates in empty chamber ($M ==$) ² .	Destroys temporary state components; evaluates enemy entity counts and reserve inventory against victory/loss conditions ⁸ .	Transitions to STATE_VICTORY, STATE_GAME_OV ER, or returns to STATE_ORBITAL_I DLE ⁸ .

Ergonomic One-Thumb Mobile Interface Architecture

Mobile interaction analysis demonstrates that single-handed vertical portrait usage accounts for roughly 67% of user sessions²². In this physical grip orientation, the user's thumb anchors at the base of the device, creating an arc of natural reach concentrated across the lower third of the display²².

Stretching the thumb into the upper screen corners induces physical strain and compromises grip stability²³. *Void Sower* structures its entire user interface to place all mechanical controls within this natural thumb sweep zone, reserving the upper regions of the display for tactical flight paths and visual telemetry¹.

The viewport is divided into three ergonomic functional tiers:

The upper 60% of the display constitutes the Tactical Combat Corridor, an unobstructed region where hostile formations, boss health bars, and atmospheric horizons are rendered¹. No interactive touch targets exist in this zone, preventing the player's thumb from occluding incoming threats and bullet patterns²³.

The middle 10% functions as the Dynamic Projection Shelf, an informational boundary displaying projected cross-discharge trajectories, impending flak impact footprints, and mathematical damage multipliers¹.

The lower 30% forms the Primary Thumb Command Arc, housing the sixteen capacitor bays, the central reactor core, and the horizontal alignment slider¹. This lower zone aligns with the comfortable reach radius (45–75 mm from the bottom corners), enabling sustained play without joint fatigue²².

UI Subsystem Component	Screen Viewport Placement	Touch Input Mapping	Visual & Haptic Telemetry Response
---------------------------	------------------------------	------------------------	--

Tactical Viewport	Upper 60% of display	Non-interactive visual display	Renders descending formations, atmospheric entries, and shield barrier health ¹ .
Projection Shelf	Middle 10% of display	Non-interactive visual display	Displays prospective cross-capture reticles and damage readouts ($\times M^2$) ¹ .
Orbital Platform Shelf	Lower 30% (Upper Arc)	Horizontal thumb drag	Slides the dreadnought laterally along the orbital horizon to align batteries with corridors ¹ .
Capacitor Bay Ring	Lower 30% (Radial Ring)	Tap & horizontal swipe	Selects origin bay; left sweep sets counter-clockwise (-1), right sweep sets clockwise ($+1$) ¹ .
Reactor Core Hub	Lower 30% (Center Base)	Upward flick gesture	Injects plasma core (<i>namua</i>), initiating automated distribution and locking manual inputs ¹ .
Tactile Engine	Hardware haptic actuator	Triggered via simulation state	High-frequency ticks accompany ring traversal; heavy transient impulse marks cross-discharge ¹ .

Platform navigation and weapon distribution are operated via three thumb gestures executed entirely within the lower command arc¹:

Horizontal alignment is executed by sliding the thumb horizontally across the lower shelf¹. This pans the dreadnought laterally along the orbital plane, bringing specific frontline batteries into alignment with descending assault formations¹. The input coordinates drive `DreadnoughtStateComponent::target_position_x` directly, with smoothing applied via critically damped spring interpolation.

Trajectory selection is executed by tapping an active capacitor bay and swiping outward along the desired angular path¹. Sweeping right sets a clockwise sowing vector (

`step_direction = +1`), while sweeping left sets a counter-clockwise vector (

`step_direction = -1`)¹. The Fluorite rendering layer responds by drawing illuminated flow vectors around the dreadnought hull, confirming the planned distribution trajectory¹.

Core injection—translating the traditional *namua* placement—is executed via an upward flick originating from the central reactor hub toward the chosen capacitor bay¹. This gesture injects a plasma core, increments the bay's mass, empties the chamber, and triggers the automated sowing sequence¹. Manual controls lock while the cascade resolves, ensuring the player can observe the resulting chain reaction¹.

To reduce cognitive friction when calculating multi-lap distributions under real-time pressure, the interface provides predictive targeting telemetry¹. Holding a thumb over an active bay triggers an instantaneous dry-run simulation in the C++ core via FFI¹. If the distribution resolves in a frontline battery aligned with an enemy column, the system illuminates the target corridor with a high-contrast reticle and displays the projected quadratic damage value directly above the threatened vessel¹.

Tactile feedback reinforces this visual pacing¹. As the relay sequence deposits plasma units into consecutive bays, the device fires light haptic impulses (`15 ms` at `30 Hz`) tracing the circular cadence¹. Multi-lap cascades trigger rising vibration frequencies that culminate in a heavy resonant transient (`80 ms`) the moment a cross-discharge particle lance fires¹.

Procedural Wave Generation and Solvability

Guarantees

Purely random enemy wave generation in a mechanics-driven tactical shooter inevitably yields unwinnable scenarios or trivial configurations that undermine player engagement⁸. If an assault wave descends with shield thresholds exceeding the quadratic damage potential of the player's remaining cores, failure is guaranteed regardless of player skill⁸.

To guarantee that every procedural encounter is mathematically solvable, *Void Sower* adapts the backward-play program inversion model and Monte Carlo Tree Search (MCTS) difficulty grading architecture established in *Oware 2048*².

The wave generation pipeline begins at the terminal victory state and recursively applies the

inverse of the game's mechanics to derive the initial encounter layout². The generator initializes an empty victory state: the sector flagship and assault wings are destroyed, the dreadnought rests at its baseline coordinate, and all capacitor bays are completely drained ($M_i = 0$ for all $i \in [0, 15]$)⁸.

Next, the algorithm executes inverse cross-discharges². Selecting a frontline battery ($i \in [8, 15]$), it applies the reverse of the beam firing equation: an enemy vessel is instantiated at the top of the aligned corridor, with combined shield and hull health calibrated to match the quadratic damage output $D = \alpha \cdot M^2$ ². The firing bay's charge is restored from zero to mass M ⁸.

Following beam inversion, the engine performs inverse sowing (gathering)⁸. Choosing an angular direction, the algorithm steps backward along the circular ring buffer:

$$i_{\text{prev}} = (i - \text{direction} \cdot k + 16) \pmod{16}$$

At each step, it decrements one unit of mass from the traversed bay and gathers it into the designated origin bay, exactly reversing the forward deposit logic⁸. Where intermediate flak bursts occurred during this path, secondary escort drones are placed at the corresponding spatial coordinates².

Finally, the generator executes inverse injection (*un-namua*)². The accumulated plasma mass within the origin bay is decremented by 1, and the extracted core is added back into the player's unplaced reserve inventory².

Entropy is introduced by placing electromagnetic hazards or spatial anomalies across non-critical bays, narrowing the forward solution down specific geometric pathways⁸. Because every operational step in this pipeline represents the inverse of a deterministic forward rule, the generated combat encounter is mathematically guaranteed to have at least one viable forward clearing path⁸.

Following level construction, encounter difficulty is graded using Monte Carlo Tree Search (MCTS) as an automated playtester². Running on background Dart isolates, the MCTS agent executes thousands of rapid game simulations using thread-local EnTT registries, sidestepping mutex locks and preventing frame drops on the main UI thread⁸. The agent explores the decision tree using the Upper Confidence Bound applied to Trees (UCT) formula⁸:

$$UCT(s, a) = Q(s, a) + c \cdot \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

where $Q(s, a)$ denotes the tactical payout of action a in state s (measured by fleet damage

achieved and cores conserved), $N(s)$ is the visit count of state s , $N(s, a)$ is the visit count of move a , and $c = \sqrt{2}$ balances tactical exploitation against exploratory branching⁸. Difficulty is quantified through MCTS convergence complexity, defined as the minimum iteration count required for the UCT algorithm to converge upon the optimal winning path⁵. Levels are classified into distinct progression tiers based on this metric, ensuring a smooth, deliberate difficulty curve across the campaign⁸.

Sector Archetype Tier	MCTS Convergence Complexity	Decision State Branching	Operational Tactical Profile
Sector Patrol (Tier 1)	$< 10^5$ Iterations ⁵ .	$B \approx 10^3$ viable branches; forgiving spatial margins ⁸ .	Multiple injection points trigger clearing cascades; low enemy barrier resistance; teaches core mechanics ⁸ .
Planetary Siege (Tier 2)	$250-1,200 \times 10^5$ Iterations ⁵ .	$B \approx 10^4$ viable branches; tight alignment windows ⁸ .	Demands deliberate relay setups to break heavy cruiser shields; flak paths must intercept incoming drone waves ⁸ .
Flagship Bastion (Tier 3)	$> 10^6$ Iterations ⁵ .	$B \leq 10^5$ viable branches; zero margin for misaligned sows ⁸ .	Complex multi-lap cascades required; improper early injections deplete core reserves, causing inevitable failure ⁸ .

Frame Budget, Spatial Indexing, and Mobile Feasibility

Achieving consistent 60 FPS performance (a 16.66 ms frame window) on mid-tier mobile hardware (ARM Mali-G57, Adreno 610) and 120 FPS (8.33 ms) on high-end chipsets requires strict resource budgeting across every engine subsystem².

```

C++
// 2D uniform spatial grid for bounding volume queries
struct SpatialGridBucket {
    static constexpr uint16_t MAX_OCCUPANTS = 16
    uint32_t entity_ids[MAX_OCCUPANTS]
    uint16_t count
}

```

To prevent frame drops during intense combat, proximity queries and beam-to-hull collision evaluations are accelerated using a 2D uniform spatial grid². The combat arena is divided into spatial buckets matching the eight attack corridors¹. During each physics tick, descending craft register their bounding volumes into corresponding bucket arrays². When a particle lance discharges, collision raycasts query only the entities registered within that specific corridor bucket². This reduces pairwise intersection testing from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, maintaining stable execution times regardless of active on-screen craft².

Subsystem Execution Phase	Fixed Frame Budget Allocation (16.66 ms)	Memory Strategy & Architecture	Profiling Metric & Objective
Input Capture & FFI Bridge	< 8 .	Zero-copy shared memory structs; direct native function pointers ⁸ .	Zero invocation latency; eliminates platform channel serialization overhead ⁸ .
ECS Simulation Loop	< 8 .	Contiguous POD sparse set iteration; branchless modulo 16 arithmetic ² .	100% L1 cache line pre-fetching; eliminates pointer-chasing latency ⁸ .
Spatial Grid & Collisions	< 2 .	2D uniform corridor bucketing pre-allocated in static memory ² .	$\mathcal{O}(N)$ raycast culling; zero runtime dynamic allocations ² .

Filament Rendering Pass	< ² .	Pre-compiled shaders via matc; pre-allocated vertex buffers in FluoriteView ² .	Stable draw-call submission; GPU uniform updates without CPU re-tessellation ¹² .
Flutter UI Compositing	< ⁸ .	Unmanaged memory inspection; repaints scoped strictly to dirty HUD nodes ⁸ .	60/120 FPS UI compositing without triggering Dart GC pauses ⁸ .
Safety & Thermal Margin	1.16 ms ¹⁴ .	Dynamic resolution scaling via AMD FSR within Filament pipeline ¹⁴ .	Prevents mobile GPU thermal throttling during extended sessions ¹⁴ .

Technical Synthesis

Void Sower: Bao Orbital Batteries translates the mathematical foundations of *Bao la Kiswahili* into an orbital defense shooter¹. By translating *namua* piece placement into single-action plasma core injections, and multi-lap relay captures into rolling flak cascades and quadratic particle lances, the game replaces reflexive twitch shooting with spatial forward calculation¹. The production-proven technology stack—combining a C++ EnTT ECS core with the Fluorite engine and Google's Filament renderer through Dart FFI shared-memory pointers—delivers console-grade visual fidelity and zero-copy state evaluations while eliminating garbage collection pauses on mobile hardware².

Placing all operational controls within the natural one-thumb sweep arc ensures comfortable, single-handed portrait play¹. Furthermore, procedural wave generation via backward-play program inversion, paired with offline Monte Carlo Tree Search difficulty grading, guarantees that every encounter contains a verified, mathematically solvable combat solution². This establishes an authentic, mechanically rigorous mobile experience that honors its indigenous ludic heritage¹.

Works cited

1. African-Inspired Mobile Shooter Concepts, https://mail.google.com/mail/?extsrc=sync&client=h&plid=ACUX6DPj1JdcDM1J_44oKWgtcGPUJx9iF46oaYk&mid=1a09b425b773526e
2. African-Inspired Mobile Shooter Concepts.pdf
3. Vintage African Heavy Wood Game Board Bao Mancala Carved, <https://www.ebay.co.uk/itm/187719765749>

4. Oware, Bao & Kalah: Regional Mancala Traditions Across Africa and, <https://ancientgames.org/oware-bao-and-kalah-regional-mancala-traditions-around-the-world/>
5. Bao Game - History, Rules, Terminology & More, <https://unitedrepublicoftanzania.com/tanzania-culture/tanzania-entertainment-news/popular-sports-tanzania/bao-game-history-rules-terminology-board-game-online-for-sale-mancala-zanzibar-buy-electronic-play-omweso-strategy-instructions-how-to-play-lulu-african-buy-app-strategy-malawi-night-in-english-kenya/>
6. Bao - traditional mancala board game - Rules and strategy of, https://gambiter.com/mancala/Bao_mancala_game.html
7. Learn how to play Bao la Kiswahili step-by-step. An evolutionary, <https://medium.com/@navpil/learn-how-to-play-bao-la-kiswahili-step-by-step-an-evolutionary-approach-7f1131bc3e0c>
8. [Games Project] Oware 2048 Technical Design Document, <https://mail.google.com/mail/?extsrc=sync&client=h&plid=ACUX6DM5eekv5Sx2m6JCZyVtvr201CqgXjADs&mid=19fbe5fb8d8e7913>
9. Oware 2048 Technical Design Document, <https://drive.google.com/open?id=1iwMis6i4J2hteWyU8inXbp7OqI9uuK95ZggaWRoJ3GU>
10. Oware-2048-Technical-Design-Document.pdf
11. African-Inspired Mobile Shooter Concepts, https://drive.google.com/open?id=1v-qNyNZB-7mKLzCYF2_GclnKL8RiOfWi4bsGN5HBLU8
12. When Flutter Hits the 3D Wall — Fluorite - Medium, <https://medium.com/flutter-taipei/when-flutter-hits-the-3d-wall-fluorite-9fb3d22c54eb>
13. RELEASE_NOTES.md - google/filament - GitHub, https://github.com/google/filament/blob/main/RELEASE_NOTES.md
14. Filament is a real-time physically based rendering engine ... - GitHub, <https://github.com/google/filament>
15. [Personal Projects] Hybrid Game Proposal, <https://mail.google.com/mail/?extsrc=sync&client=h&plid=ACUX6DPUKRzDcoIEE9TwsHJAfoF5oUOPF-eLIDc&mid=19fbd959f7844e71>
16. Bao (game) - Wikipedia, [https://en.wikipedia.org/wiki/Bao_\(game\)](https://en.wikipedia.org/wiki/Bao_(game))
17. Changing material for glb at runtime. · google filament - GitHub, <https://github.com/google/filament/discussions/6431>
18. Bao: East Africa's Four-Row Mancala Rules and Context, <https://toguzkumalak.com/en/blog/posts/bao-east-african-mancala.html>
19. Bao la Kiswahili - App Store - Apple, <https://apps.apple.com/no/app/bao-bao-la-kiswahili/id6799564167>
20. Game | Sheikh Jahbooty, <https://sheikhjahbooty.wordpress.com/tag/game/>
21. Your Day Ahead 08/02/2026, https://mail.google.com/mail/?extsrc=sync&client=h&plid=ACUX6DPwgv_wL7GiUWQ3uoDmR4R57RY6bOQikR4&mid=19fc2237313e8a4e
22. How Do Users Really Hold Mobile Devices? - UXmatters,

<https://www.uxmatters.com/mt/archives/2013/02/how-do-users-really-hold-mobile-devices.php>

23. Mastering the Thumb Zone: Mobile UX & UI Design Guide,
<https://parachutedesign.ca/blog/thumb-zone-ux/>
24. Creating a Thumb-Friendly Experience: Designing Mobile Websites,
<https://rizzoyoung.com/digital-marketing-blog/creating-a-thumb-friendly-experience-designing-mobile-websites>
25. Hybrid-Mobile-Game-Proposal.pdf